



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE



Fundamentals of Software Architecture: A Modern Engineering Approach

Mark Richards and Neal Ford—hands-on practitioners who have taught software architecture classes professionally for years—focus on architecture principles that apply across all technology stacks. You'll explore software architecture in a modern light, taking into account all the innovations of the past decade.

O'REILLY®

Second
Edition



Fundamentals of Software Architecture

A Modern Engineering Approach

Mark Richards & Neal Ford





Design e Arquitetura de Software: Uma Jornada Evolutiva

Nesta aula, vamos explorar a importância da arquitetura de software, desvendando sua evolução histórica e a influência dos estilos arquiteturais nas práticas de desenvolvimento modernas.



Warm up

O design e a arquitetura de software servem como **modelo para o desenvolvimento de software**

Fornecem uma **visão de alto nível do sistema**, descrevendo a **estrutura, o comportamento e as interações** dos componentes de software.

Este design **orienta** os engenheiros e engenheiras de software em seu trabalho, ajudando-os a criar sistemas robustos, eficientes e adaptáveis.



Compreendendo o Design de Software

Envolve o planejamento, criação e especificação da estrutura arquitetural de um sistema de software

É o processo de transformar requisitos em um plano que guia a implementação e manutenção desse sistema

Ao aderir aos princípios-chave de design, os arquitetos de software podem traduzir de forma eficaz os requisitos em arquiteturas de software robustas e eficientes



O que é Arquitetura de Software?

Concentra-se na organização de alto nível do sistema

É o processo de projetar e organizar a estrutura de um sistema de software

Ou seja, envolve a tomada de decisões de alto nível sobre como os diferentes componentes do sistema irão interagir uns com os outros e como eles serão organizados.



Habilidades interpessoais de um arquiteto

Espera-se que um arquiteto possua habilidades interpessoais excepcionais, incluindo trabalho em equipe, facilitação e liderança.

Como tecnólogos, desenvolvedores e arquitetos gostam de resolver problemas técnicos, não problemas de pessoas.

Um arquiteto não deve apenas fornecer orientação técnica à equipe, mas também liderar as equipes de desenvolvimento por meio da implementação da arquitetura. As habilidades de liderança representam pelo menos metade do que é preciso para se tornar um arquiteto de software eficaz, independentemente da função ou título que o arquiteto possua.

Outras habilidades interpessoais que um arquiteto deve possuir incluem:

- Capacidade de tomar decisões de arquitetura e princípios de design para orientar as decisões tecnológicas dentro da equipe.
- Capacidade de analisar continuamente a arquitetura e o ambiente tecnológico atual e, em seguida, recomendar soluções para melhorias.
- Ter uma exposição diversificada a várias tecnologias, estruturas, plataformas e ambientes.
- Possuir conhecimento do domínio de negócios.

Um arquiteto de software eficaz deve não apenas entender os aspectos técnicos da arquitetura de software, mas também as principais técnicas e habilidades sociais necessárias para pensar como um arquiteto, orientar as equipes de desenvolvimento e comunicar efetivamente a arquitetura para vários interessados.





Estilos Arquitetônicos: Tipos, Forças, Fraquezas e Insights



O que são estilos arquiteturais

Os estilos arquiteturais representam **estruturas gerais de organização do código e interação com bancos de dados.**

Eles encapsulam um **conjunto de trade-offs** bem conhecidos que ajudam arquitetos a tomar decisões adequadas para problemas específicos



Padrões fundamentais

Os **Padrões Fundamentais** são **modelos arquiteturais básicos** que servem como alicerces para a construção de sistemas de software. Eles representam **estruturas comuns** de organização do código e da interação entre os componentes do sistema, ajudando arquitetos a tomar decisões estruturais iniciais.



Big Ball of Mud

O Big Ball of Mud é um "anti-padrão" arquitetural caracterizado por:

- Código desorganizado e sem uma estrutura clara.
- Acoplamento excessivo entre componentes e baixa coesão.
- Dificuldade na manutenção e evolução do software.
- Frequentemente visto em sistemas que crescem de forma descontrolada ao longo do tempo.

Pode emergir quando **equipes trabalham sob alta pressão e sem um planejamento adequado**. Para evitar isso, boas práticas como **modularização, separação de responsabilidades e adoção de frameworks** adequados são essenciais.

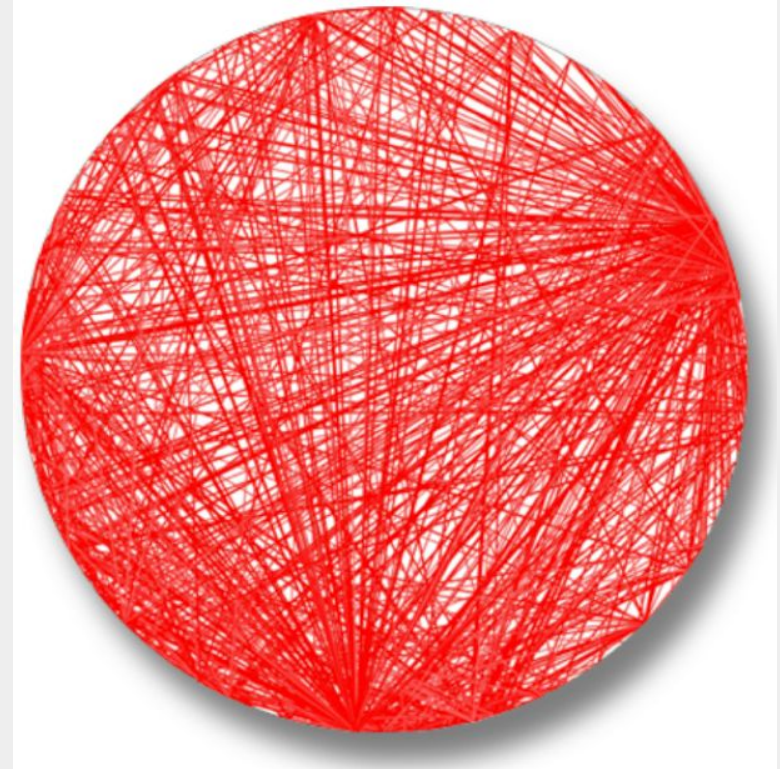


Figure 9-1. A Big Ball of Mud architecture visualized from a real code base



Unitary Architecture

A Unitary Architecture (Arquitetura Unitária) representa um modelo de arquitetura extremamente simples, onde:

- Todo o código é executado dentro de um único processo.
- A lógica de negócios, UI e armazenamento de dados são unificados em um único ambiente.
- Normalmente usada em aplicações pequenas e embarcadas.

Embora limitada, a arquitetura unitária pode ser **ideal para aplicações simples**, como softwares embarcados, sistemas monousuário ou pequenos scripts utilitários.



Client/Server

O modelo Cliente/Servidor é uma arquitetura mais estruturada, dividindo a aplicação em duas partes:

- **Cliente:** Interface com o usuário, responsável pela entrada e exibição dos dados.
- **Servidor:** Lógica de negócios e manipulação de dados, geralmente acessados através de uma API.

Essa separação permite que o cliente e o servidor **sejam desenvolvidos independentemente**, facilitando a modularidade do sistema.

O modelo Cliente/Servidor é a base de muitas arquiteturas modernas, como aplicações web e APIs RESTful, onde múltiplos clientes (navegadores, apps móveis) acessam um backend centralizado.



Resumo - Padrões fundamentais

Padrão	Características Principais	Pontos Fortes	Pontos Fracos
Big Ball of Mud	Código desorganizado, acoplado, difícil de manter.	Implementação rápida, flexibilidade.	Manutenção difícil, baixa escalabilidade, alto risco técnico.
Unitary Architecture	Simplicidade extrema, tudo em um único ambiente.	Fácil de implementar, custo reduzido.	Difícil de escalar, baixa modularidade.
Client/Server	Separação entre cliente e servidor, comunicação via rede.	Melhor modularidade, escalável, flexível.	Maior complexidade, depende de rede, sobrecarga no servidor.



Estilos Arquiteturais

Os estilos de arquitetura podem ser classificados em dois tipos principais: **monolíticos** (unidade de implantação única de todo o código) e **distribuídos** (várias unidades de implantação conectadas por meio de protocolos de acesso remoto).

Monolíticos

Arquitetura em Camadas

Pipeline

Microkernel

Distribuídos

Baseada em Serviços

Orientada a Serviços

Dirigida a Eventos

Microserviços

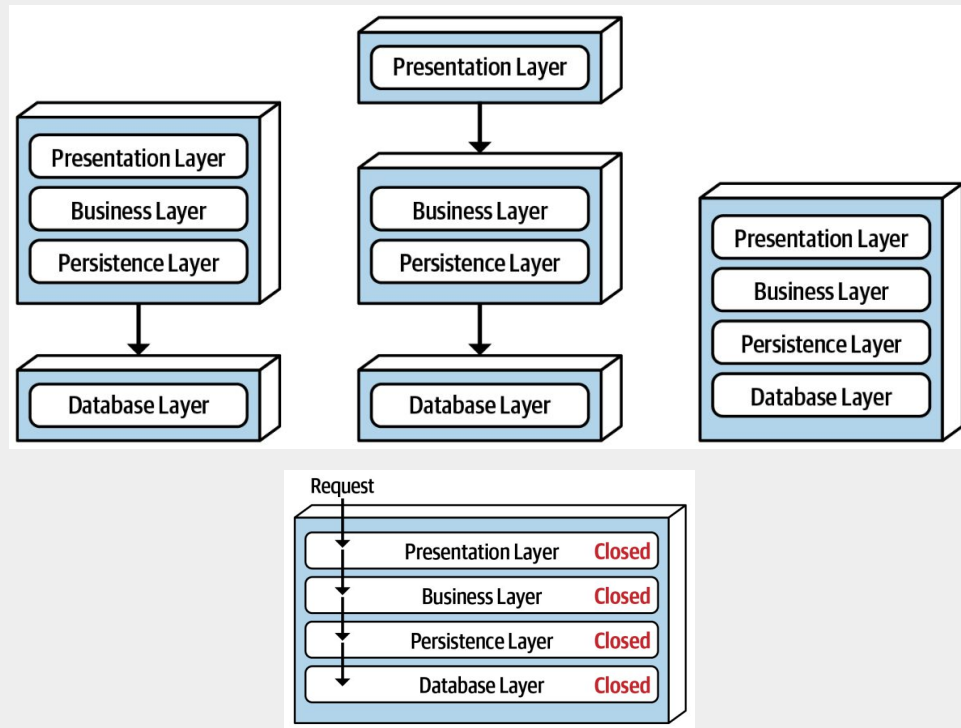
Baseada em Espaços



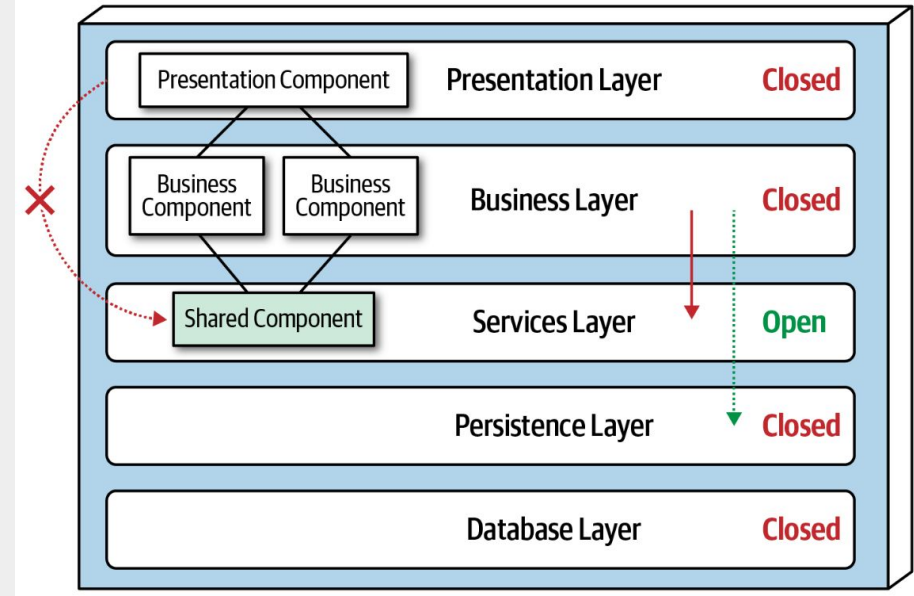
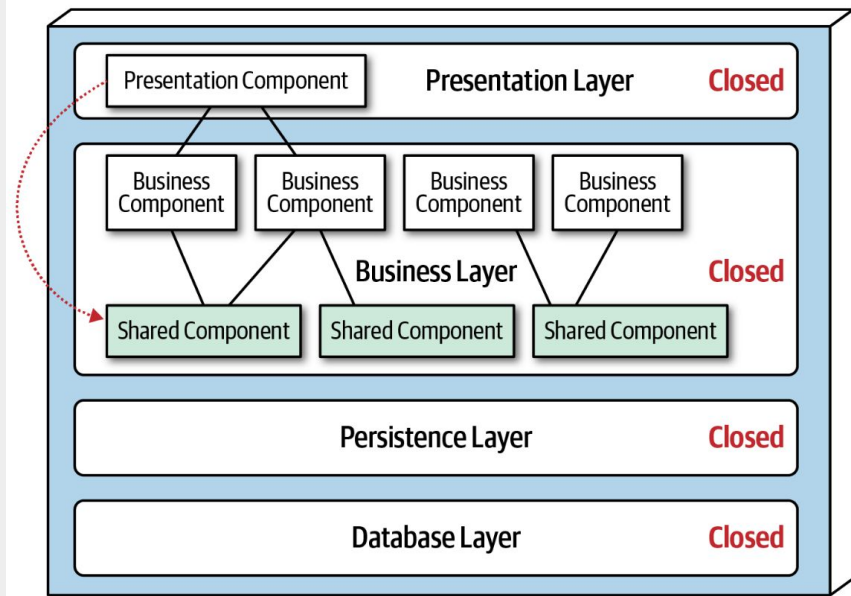
Arquitetura em Camadas

A Arquitetura em Camadas é um dos estilos arquiteturais mais comuns, onde o sistema é dividido em camadas horizontais, cada uma com uma responsabilidade bem definida.

- ★ **Facilidade de manutenção:** Separação clara de responsabilidades.
- ★ **Boa organização:** Código mais estruturado e modular.
- ★ **Flexibilidade:** Facilita mudanças em partes específicas do sistema sem impactar as demais.
- **Baixo desempenho em grandes sistemas:** Pode gerar muitas camadas intermediárias desnecessárias.
- **Dificuldade de escalabilidade:** Camadas podem se tornar gargalos de desempenho.
- **Risco de violações:** Desenvolvedores podem ignorar camadas e criar dependências indesejadas.



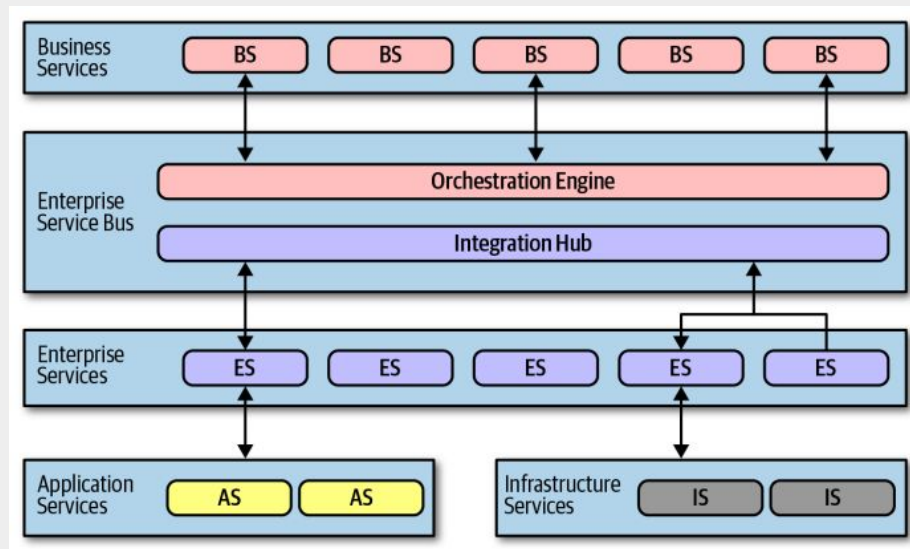
Adicionando camadas



Arquitetura Orientada a Serviços (SOA)

A Service-Oriented Architecture (SOA) é um estilo arquitetural onde um orquestrador central controla a comunicação e execução dos serviços, garantindo que eles sejam chamados na ordem correta e de forma coordenada.

- Orquestrador central (como um ESB – Enterprise Service Bus) gerencia a lógica do fluxo de serviços.
- Serviços independentes, mas coordenados pelo orquestrador.
- Alta governança e controle sobre chamadas e interações entre serviços.
- Suporte a workflows complexos, garantindo que processos sigam regras pré-definidas.
- Fluxo de processos bem definido: O orquestrador garante que todas as etapas sejam executadas corretamente.
- Facilidade de monitoramento e depuração: Como há um controle central, é mais fácil rastrear problemas.
- Integração facilitada: Diferentes serviços podem ser gerenciados por um único ponto de controle.
- Ponto único de falha: Se o orquestrador falhar, todo o sistema pode ser afetado.
- Acoplamento centralizado: Serviços ficam dependentes do orquestrador, reduzindo autonomia.
- Escalabilidade limitada: O orquestrador pode se tornar um gargalo à medida que a carga aumenta.



Vocabulário: mesma palavra, significados diferentes

Palavra	Contexto	O que significa
service	Pasta services/ do Compasso	Camada de lógica de negócio dentro do monolito (ex.: TimeEntryService)
	SOA	Unidade autônoma do sistema, orquestrada por um ESB
	Microserviços	Processo independente, com seu próprio banco de dados
component	React/Vite	Bloco de UI reutilizável
	C4 Nível 3	Módulo dentro de um contêiner
contêiner	C4 Nível 2	Qualquer processo em execução independente (ex.: API Node.js)
	Docker/DevOps	Ambiente de execução isolado

Mesma palavra, outro mundo. Sempre pergunte: **em que contexto?**

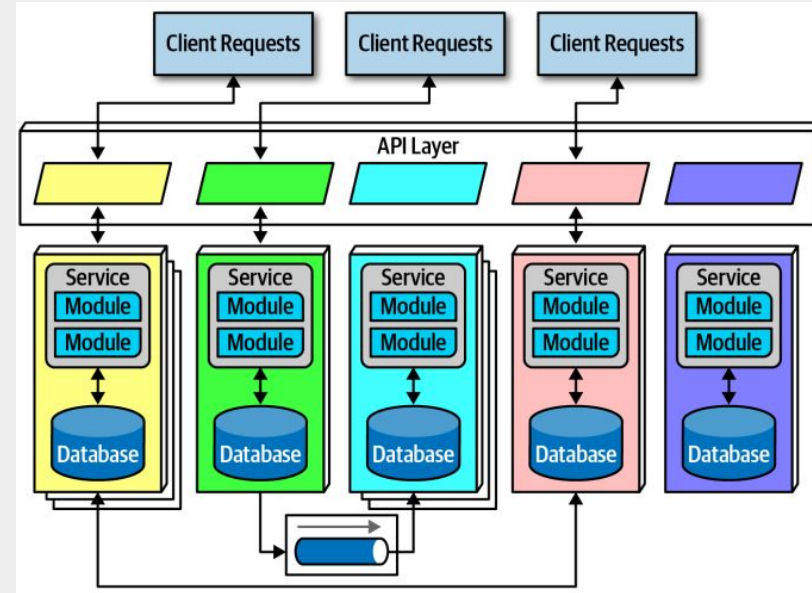


Microservices Architecture

É um estilo de arquitetura que estrutura uma aplicação como uma coleção de serviços pequenos e independentes que são executados em seu próprio processo e se comunicam com mecanismos leves, muitas vezes uma API HTTP. Cada microserviço é “pequeno” e se concentra em fazer uma coisa bem.

Benefícios?

Podem levar a uma maior complexidade de gerenciamento, pois você tem muitos serviços diferentes para monitorar e gerenciar. Além disso, a comunicação entre microserviços pode ser complexa e a consistência de dados entre serviços pode ser um desafio.



Bounded Context (DDD): a ver na Semana 11

Bounded Context é um conceito do **Domain-Driven Design (DDD)**: o limite dentro do qual um modelo de domínio e seus dados fazem sentido.

Por ora, guarde só a intuição:

cada serviço tem o seu próprio modelo e os seus próprios dados.

a ver na

Semana 11

Bounded Context e DDD

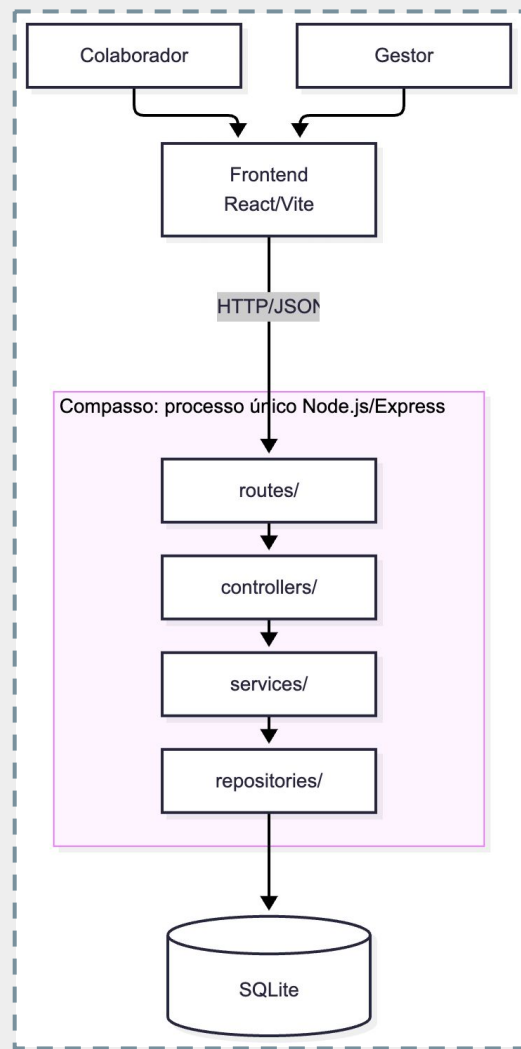


Monolito modular: o Compasso

Imagine que a equipe acabou de criar o **Compasso**, uma plataforma de time tracking.

Depois das primeiras reuniões e do levantamento de requisitos, o backend Express nasce organizado em camadas: **routes** → **controllers** → **services** → **repositories**.

Resultado: **um único processo** (um monolito modular) que atende Colaborador e Gestor.



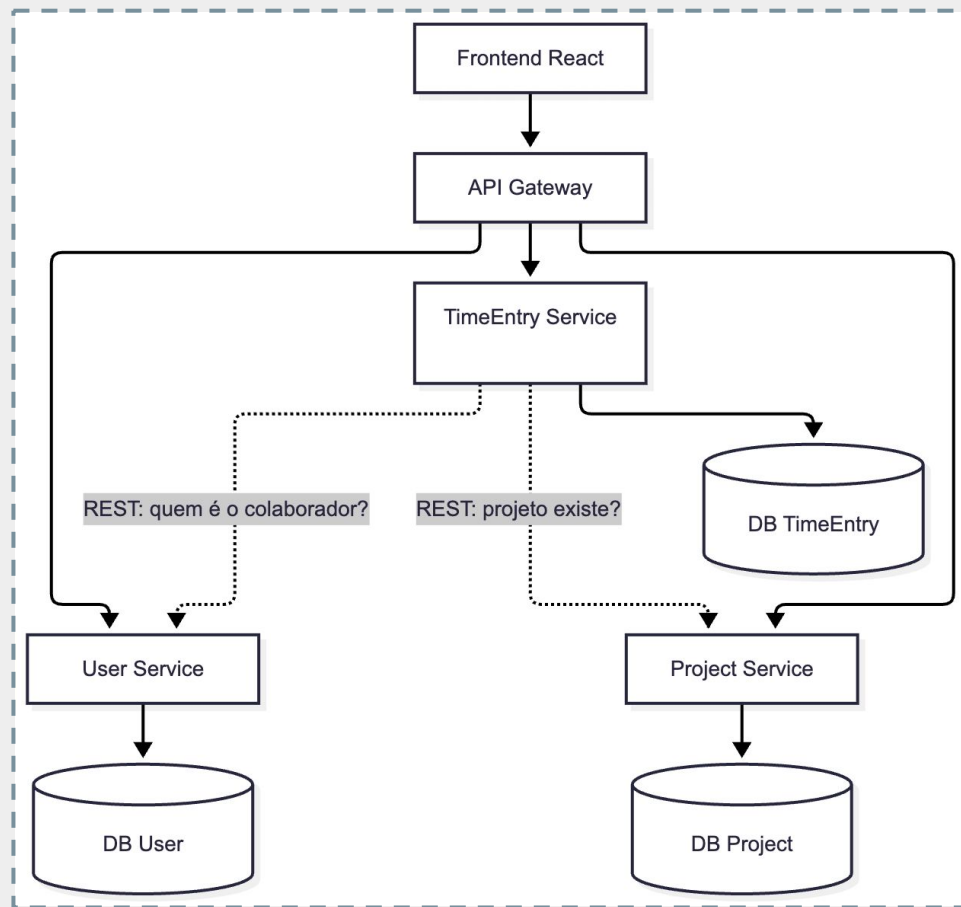
Microserviços: e se o Compasso fosse distribuído?

Hipótese (não é o que vamos construir):

Cada área funcional do Compasso vira um microserviço próprio (TimeEntry, Project e User), cada um com o seu banco de dados.

Cada serviço expõe uma API REST e consome as APIs dos outros. O frontend não fala direto com eles: passa por um **API Gateway**.

Mesmos módulos do monolito, agora como três processos, três bancos e chamadas de rede.



Por que Arquitetura de Microsserviços?

Arquitetura de microsserviços divide a funcionalidade do aplicativo entre um conjunto de microsserviços independentes, tornando o CI/CD muito mais fácil.

São mais rápidos de desenvolver e muito mais fáceis de entender e manter. Cada serviço pode ser desenvolvido separadamente por uma equipe focada apenas naquele serviço. A equipe pode escolher a linguagem de programação, a plataforma de desenvolvimento e o banco de dados que melhor se adequam ao microsserviço. Os microsserviços geralmente são empacotados como contêineres para simplificar ainda mais a implantação.

Os microsserviços são fracamente acoplados, o que significa que não dependem dos componentes internos de outros microsserviços. Em vez disso, eles se comunicam entre si via API. Desde que a API exposta por cada microsserviço permaneça compatível com versões anteriores, as alterações em um microsserviço não exigem atualizações em nenhum outro.

7 Benefits of Microservices

- 01 Load Balancing** – Safely and independently scale out individual app components using advanced load balancing
- 02 Faster Development** – Update app components on their own schedules without worrying about dependencies on other components
- 03 Flexible Deployment** – Deploy app components piece-by-piece without affecting other systems
- 04 Cache** – Improve performance while reducing load on your app with flexible content caching
- 05 Simpler** – Develop and maintain more easily
- 06 Troubleshooting** – Easily identify failures in service
- 07 Agnostic** – Run in containers, on your existing hardware, or in the cloud



NGINX
Part of F5

SOA vs. Microsserviços

São abordagens para construir aplicações a **partir de serviços independentes**.

Os microsserviços são serviços **muito finamente granulados**, separados por uma **razão funcional**. Isso significa que em uma arquitetura de microsserviços, um serviço - **incluindo o banco de dados e os servidores de aplicação** - é **gerenciado de ponta a ponta pela mesma equipe**. A SOA, por outro lado, **separa os serviços de maneira lógica**.

Os microsserviços **não são limitados** pelos mesmos frameworks de comunicação, protocolos e especificações que limitam a SOA.



Fatores Essenciais para Escolher um Estilo Arquitetural

Escalabilidade

Como o sistema irá lidar com o aumento de usuários e dados?

Arquiteturas de microsserviços e baseadas em nuvem oferecem grande escalabilidade.

Manutenibilidade

O sistema será fácil de modificar e corrigir? Arquiteturas modulares, como microsserviços e em camadas, facilitam a manutenção.

Requisitos de Negócio

Quais são as necessidades específicas do negócio? O estilo arquitetural deve atender aos requisitos de funcionalidade, desempenho e segurança.



ADR-002 — Usar monolito modular

Status

Aceito

Contexto

equipe pequena (4 pessoas), prazo curto (4 semanas) e sem necessidade de escala horizontal

Decisão

vamos adotar um monolito modular (Node.js/Express com camadas separadas), porque a equipe é pequena e o prazo é curto

Consequências

Mais fácil: um único deploy, simples de desenvolver e testar.
Mais difícil: migrar para microsserviços no futuro será custoso; aceito para este contexto.

Arquivo: docs/ADR-002.md (ao lado de docs/ADR-001.md, decisão do SQLite) • Formato Nygard



Transições entre Estilos Arquiteturais

Migração Gradual

A migração gradual envolve a implementação de novos serviços em uma nova arquitetura, enquanto a arquitetura antiga continua a funcionar. Permite uma transição suave e reduz riscos.

Migração Big Bang

A migração big bang envolve a substituição completa da arquitetura antiga pela nova de uma só vez. Mais rápida, mas com maior risco de falhas e interrupções.

Estratégias e Ferramentas

Existem diversas ferramentas e técnicas para facilitar a transição, como o uso de contêineres, APIs, e frameworks de orquestração.

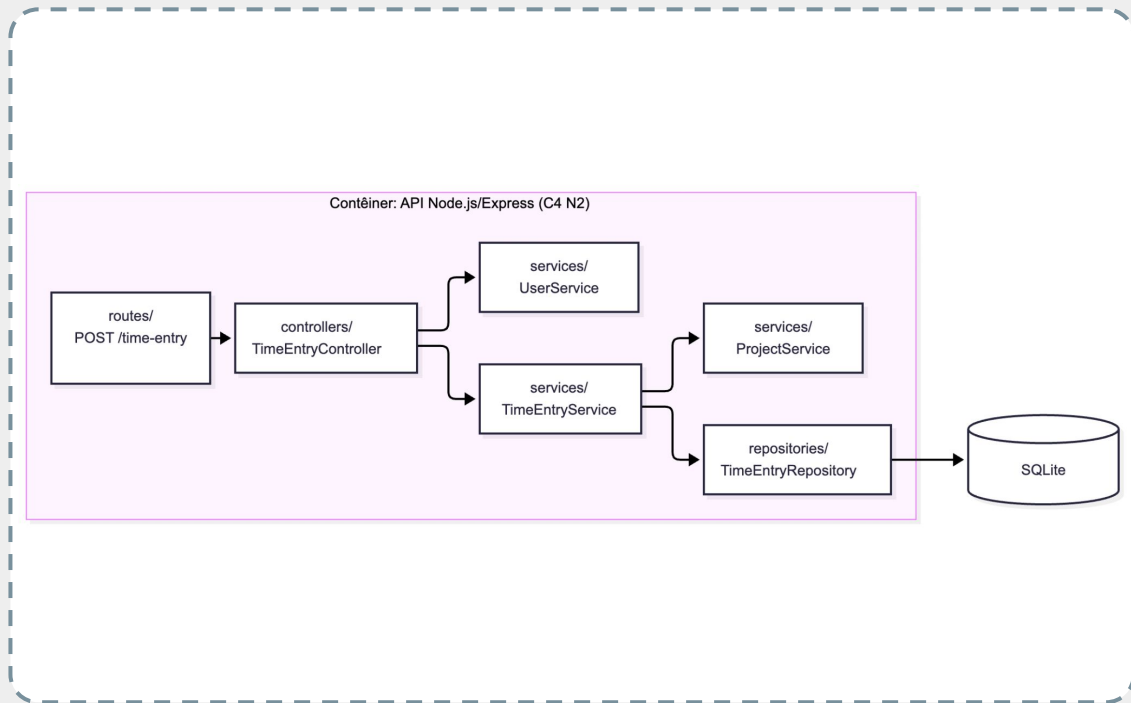


Arquitetura como Plataformas de ConversAÇÃO

A arquitetura de software facilita a comunicação entre os diferentes stakeholders envolvidos no projeto. A equipe de desenvolvimento, os gerentes de projeto, os usuários finais e os stakeholders de negócios podem entender melhor o sistema e suas funcionalidades por meio da documentação da arquitetura.



Ponte: suas pastas já são um diagrama C4



Contêiner · C4 N2

A API Node.js/Express inteira: um processo em execução (a caixa externa).

Componentes · C4 N3

Cada módulo das pastas (controller, service, repository), como TimeEntryService (as caixas internas).

Você já organiza o backend assim. Na próxima aula, cada bloco ganha nome e nível no C4.



Intro ao C4: quatro zooms sobre o mesmo sistema

N1

Contexto

Quem usa o sistema e com quais outros sistemas ele conversa?

N2

Contêineres

Quais processos em execução compõem o sistema?

N3

Componentes

Quais módulos existem dentro de um contêiner?

N4

Código

Como um componente é implementado (classes, funções)?

- O ADR registra o porquê das decisões. O C4 desenha o quê e como as partes se conectam: é a mesma conversa, em forma visual.
- **Cuidado com o vocabulário:** contêiner não é Docker e component não é React (veja o slide de vocabulário desta aula).
- **Para a próxima aula:** leiam o Cap. 14 e levem um esboço do projeto de vocês, com atores e blocos técnicos (o Compasso é o modelo).



Arquiteturas não são estáticas

As arquiteturas não são estáticas, mas evoluem ao longo de longos períodos à medida que as tecnologias amadurecem.

Eles também evoluem durante o curso normal de projetar um sistema individual.

Esses padrões evolutivos são úteis para entender a importância da arquitetura.



Leis da Arquitetura de Software

Embora o escopo da arquitetura de software seja quase impossivelmente amplo, existem elementos unificadores.

Vivemos tropeçando na Primeira Lei da Arquitetura de Software: **Tudo na arquitetura de software é um trade-off.**

Nada existe em um espectro bom e limpo para arquitetos de software. Toda decisão deve levar em conta muitos fatores opostos.

Se um arquiteto pensa que descobriu algo que não é um trade-off, é mais provável que ele ainda não tenha identificado o trade-off. —

Corolário 1



Leis da Arquitetura de Software

Definimos a arquitetura de software em termos além do andaime estrutural, incorporando princípios, características e assim por diante.

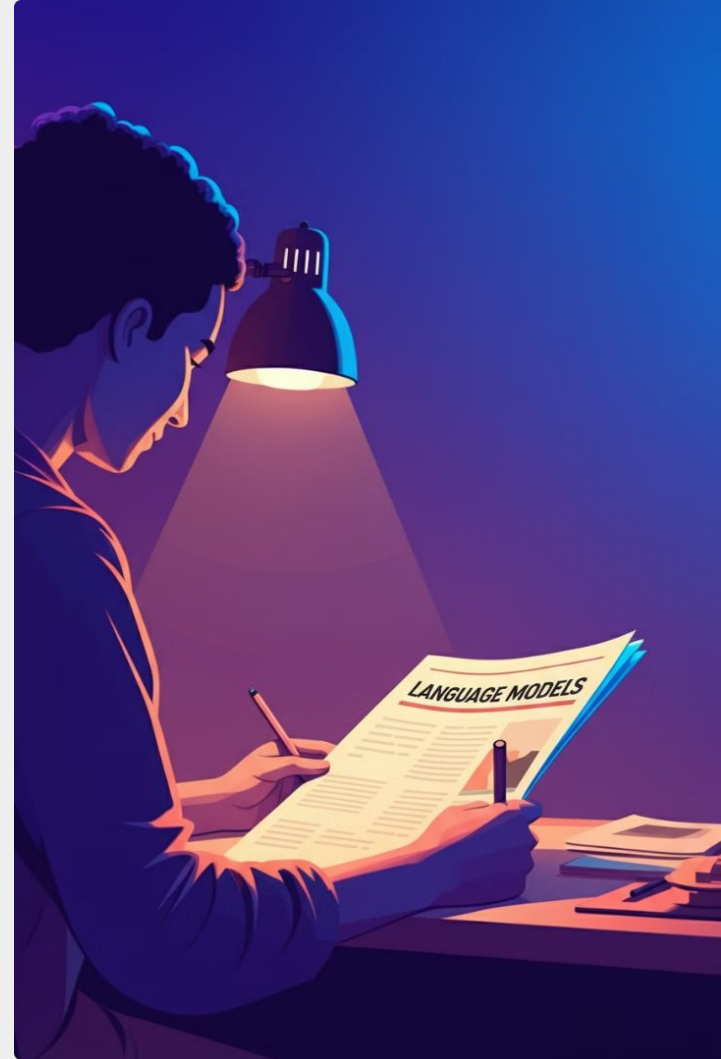
A arquitetura é mais ampla do que apenas a combinação de elementos estruturais, refletida em nossa **Segunda Lei da Arquitetura de Software: Por que é mais importante do que como.**

Um arquiteto pode olhar para um sistema existente do qual não tem conhecimento e verificar como a estrutura da arquitetura funciona, mas terá dificuldade em explicar **por que certas escolhas foram feitas em detrimento de outras.**



Próximos passos

- Eng. Software em Dimensões
 - Cap. 14 (seções 14.1–14.8):
Arquitetura, C4 Model,
qualidade arquitetural
- Eng. Software Moderna
 - Cap. 5 — Princípios de Projeto,
Coesão, acoplamento, SOLID
 - Cap. 7 — ArquiteturaCamadas,
MVC, trade-offs



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
[viniciusgarcia.me](https://www.viniciusgarcia.me)

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Design e Arquitetura de Software